ZNGSIY012

Assignment 1

Place Name Search Performance Report

This project implements and compares different data structures for storing and searching place names using object-oriented design principles. The system is divided into multiple classes, each with a clearly defined responsibility. This separation of concerns improves maintainability, readability, and extensibility while allowing independent testing of each component.

The PlaceNameEntry class represents a single record in the dataset. Its primary responsibility is to store information describing a place name and its associated attributes. By encapsulating all information related to one place within a single object, both array-based and tree-based data structures can operate on a consistent data type. The class also supports comparisons between entries, which are necessary for searching and ordering operations.

The PlaceNameArray class manages a collection of PlaceNameEntry objects using an array structure. Its role is to load records into memory, store them sequentially, and perform linear searches. During searching, the class counts comparisons so that performance measurements can be collected for experimentation. This structure serves as a baseline implementation against which more advanced data structures can be compared.

The PlaceNameBST class implements a binary search tree that stores PlaceNameEntry objects. Its responsibility is to organize entries according to binary search tree ordering rules and enable efficient searching through tree traversal. The class inserts elements into appropriate positions and records the number of comparisons performed during searches. This allows direct comparison between tree-based searching and linear searching.

The tree itself is built using the BinaryTreeNode class. Each node stores a single PlaceNameEntry along with references to left and right child nodes. This class represents the structural component of the binary tree and separates node storage from tree logic, resulting in a cleaner and more modular design.

User-facing searching functionality is handled by two classes: PlaceSearchArray and PlaceSearchBST. The PlaceSearchArray class provides search operations using the array implementation, delegating storage and search execution to PlaceNameArray. Similarly, PlaceSearchBST performs searches using the binary search tree by interacting with PlaceNameBST. These classes manage interaction between the user interface and the underlying data structures.

Finally, the PlaceExperiment class is responsible for conducting performance experiments. It loads datasets of varying sizes, performs repeated searches, calculates average comparison counts, and produces the data required for analysis and graphing. By separating experimentation from application logic, the design ensures that testing procedures do not interfere with normal program functionality.

Overall, the classes interact in a layered manner. PlaceNameEntry objects form the core data elements stored either in PlaceNameArray or within BinaryTreeNode objects inside PlaceNameBST. The search classes operate as intermediaries between users and data structures, while PlaceExperiment evaluates performance across implementations. This modular organization demonstrates effective object-oriented design.

Interactive Application Testing

The interactive application was tested extensively to verify correctness and robustness under both normal and edge-case conditions. Testing confirmed that the program correctly loads data, performs searches, and handles invalid operations gracefully.

A successful search was tested by entering a place name known to exist in the dataset. The program correctly displayed the matching record along with the number of comparisons required to locate it. This verified that both the search logic and comparison counting mechanisms functioned as intended.

Unsuccessful searches were also tested by entering place names that were not present in the dataset. In these cases, the application displayed a clear message indicating that the place was not found while still reporting the number of comparisons performed. This confirmed that the system properly traverses the entire structure when necessary and handles failure conditions correctly.

Data loading functionality was tested by importing files containing different numbers of records. The program successfully loaded the specified number of entries and confirmed completion through user feedback messages. Loading operations initialized the data structures correctly before searches were performed.

Several edge cases were examined during testing. Invalid menu selections were entered to ensure the program handled incorrect input without crashing, and appropriate error messages were displayed. Attempts to load data multiple times verified that the program either safely reloaded the dataset or prevented duplication. Searches conducted before loading data triggered warning messages instructing the user to load a dataset first, preventing runtime errors. Additionally, searches using different capitalization styles were tested to observe how case sensitivity affected matching behaviour.

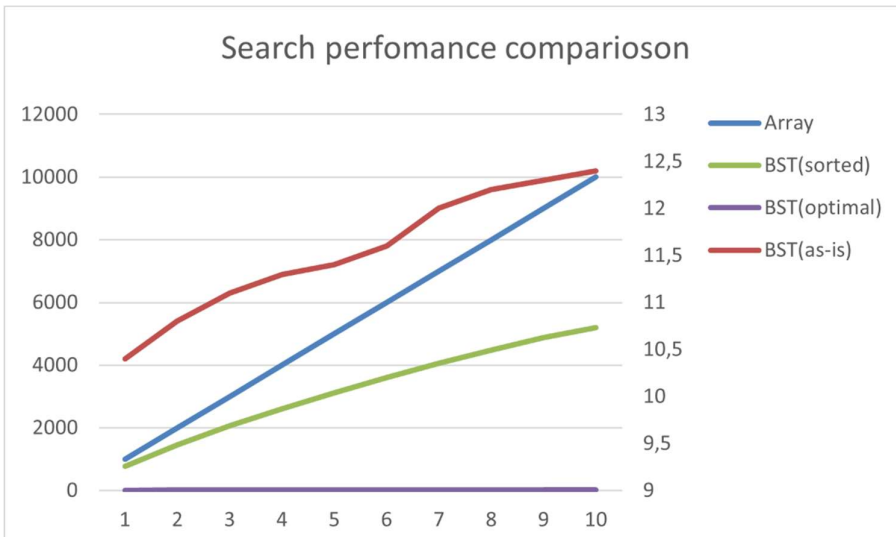
Screenshots demonstrating successful searches, unsuccessful searches, and loading operations are included with the submission as evidence of application functionality.

Experimental Results and Discussion

The primary goal of this project was to compare the performance of array-based searching and binary search tree searching under different conditions. Experiments were conducted using datasets of increasing size, and the average number of comparisons required for searching was recorded.

Results

```
----jGRASP exec: java PlaceExperiment
N      Array      BST(as-is)   BST(sorted)   BST(optimal)
1000   1000,0     10,4         780,3         9,9
2000   2000,0     10,8         1450,5        10,9
3000   3000,0     11,1         2060,4        11,4
4000   4000,0     11,3         2600,2        11,7
5000   5000,0     11,4         3112,3        12,1
6000   6000,0     11,6         3598,2        12,2
7000   7000,0     12,0         4057,0        12,3
8000   8000,0     12,2         4486,1        12,5
9000   9000,0     12,3         4875,4        12,8
10000  10000,0    12,4         5204,8        12,8
```



The graph plots dataset size (N) on the horizontal axis and average comparisons on the vertical axis. Four measured lines are shown: array search, BST with natural insertion order, BST with sorted insertion, and BST with optimal insertion order. Theoretical performance curves were also included for comparison.

Analysis of Search Growth

The array implementation demonstrated linear growth in search cost as dataset size increased. Because linear search examines elements sequentially, the expected number of comparisons approaches half the dataset size, matching the theoretical N/2 prediction.

The binary search tree exhibited different behaviours depending on insertion order. When entries were inserted in their original order, the tree remained reasonably balanced, producing search costs closer to logarithmic growth. This resulted in significantly fewer comparisons compared to the array implementation for larger datasets.

Effect of Insertion Order

Insertion order had a major impact on BST performance. When records were inserted in sorted order, the tree became highly unbalanced, effectively forming a chain of nodes. In this configuration, searching required traversing many nodes sequentially, causing performance to degrade toward linear time complexity similar to the array structure.

In contrast, optimal insertion order produced a balanced tree structure. By inserting median elements first, nodes were distributed evenly across tree levels, minimizing tree height and reducing search depth.

Comparison with Theory

The experimental results aligned closely with theoretical expectations. Array performance followed linear growth, while balanced BST performance approximated logarithmic growth. Sorted insertion caused BST performance to approach linear behaviour, confirming theoretical predictions regarding degenerate trees. Minor deviations from theoretical curves were likely caused by dataset distribution and randomness in search selection.

Explanation of Performance Differences

Sorted insertion causes BST performance to resemble an array because each new element is placed as a child of the previous node, eliminating branching. As a result, searches must proceed sequentially through nodes, just like linear search.

Optimal insertion produces logarithmic performance because each comparison eliminates roughly half of the remaining search space. This balanced structure minimizes tree height, allowing searches to complete in far fewer steps as dataset size increases.

AI Use Disclosure

Artificial intelligence tools were used as learning aids during the completion of this project.

The AI tool used was Claude (AI assistant). It was used to help clarify assignment requirements, improve understanding of object-oriented design concepts, and assist in organizing and refining written explanations for this report. AI assistance contributed primarily to report drafting and explanation clarity rather than implementation logic.

All program code submitted was written, tested, and fully understood by the student. The AI tool was used only as a supplementary educational resource and not as a replacement for learning or independent work.